

CS2610: Computer Organization and Architecture Lab

Lab 7: Advanced Trap Handling and Exception Management

16th March 2026

Problem 1: The Unified Trap Handler (30 Points)

Objective

To implement a comprehensive Machine-mode (M-mode) Trap Handler in RISC-V assembly. Students will learn to decode the `mcause` register, utilize `mtval` for fault diagnosis, and implement realistic handling logic for various synchronous exceptions including alignment errors, access faults, and environment calls.

In this lab, you will write a robust trap handler that resides in Machine Mode. Your code must capture exceptions triggered by the processor, identify the cause, and perform a "realistic" action (such as skipping the faulting instruction or logging the error state to registers) before resuming execution.

(a) Handling Specific Exceptions

Your handler must explicitly identify and handle the following `mcause` codes:

1. **Breakpoint (3):** Execution of the `ebreak` instruction.
2. **Illegal Instruction (2):** Encountering an undefined opcode.
3. **Environment Call from U-mode (8):** System call from User mode.
4. **Load Address Misaligned (4):** Target address of a load is not aligned.
5. **Load Access Fault (5):** Address not part of the process virtual address space.

(b) Requirement: Context Saving

Since a trap can occur at any point, the handler **must not** corrupt the state of the program it interrupted. You must:

- Save all registers being used in the handler to the stack (specifically `t0–t6` and any `a` registers modified).
- Use `csrr` to read `mcause`, `mepc`, and `mtval`.
- Restore the saved registers before executing `mret`.

(c) Realistic Handling Logic

For this lab, "handling" is defined as follows to ensure the system remains operational:

- **For Ecalls/Breakpoints:** Increment mepc by 4 so the program continues after the instruction. Store 0xFEED in a0 for ecalls and 0xBEEF for breakpoints.
- **For Illegal Instructions:** Log the faulting opcode from mtval into register s9, increment mepc by 4 to skip the instruction, and continue.
- **For Misaligned Loads (4):** Log the faulting address from mtval into register s10, increment mepc by 4, and continue.
- **For Access Faults (5):** Log the faulting address to s11, increment mepc by 4, and continue

Appendix: Exception Triggering Reference

To test your handler, use the following code snippets in your ucode block:

Cause	Exception	Example Trigger Codes
2	Illegal Instruction	.word 0x00000000
3	Breakpoint	ebreak
4	Load. Misaligned	la t0, _stack_low; addi t0, t0, 1; ld t0, 0(t0)
5	Load Access Fault	li t0, 0x0; ld t1, 0(t0)
8	U-mode Ecall	ecall

Architectural Framework Skeleton

Listing 1: Required Assembly Structure

```
.section .text.init
.global _start

_start:
    # 1. Initialize Stack Pointer (sp)

    # 2. Setup Trap Vectors (mtvec)

    # 3. Prepare transition to User Mode (mstatus and mepc)

    # 4. Execute mret to jump to ucode
    mret

.section .text
.align 4
mtrap_handler:
    # --- Context Saving ---
    # Save registers used in ucode. In the ideal case should save all registers.

    # --- Decode mcause ---
    # Implement logic to handle causes 2, 3, 4, 5, 8
```

```

# --- Context Restoration ---
mret

ucode:
# --- Sequence of Exception Tests ---
# Trigger exceptions one after another to test your handler logic

j .

.section .bss
.align 16
_stack_low:
    .space 4096
_stack_top:

```

Submissions

- The lab7.s file.
- Screenshots showing the 5 exceptions on spike.
- In ucode, dump the contents of s9, s10, s11 after all exceptions are handled.

Problem 2: C to Assembly Interface (20 Points)

In this problem, we will explore the **C and Risc-V assembly interface** by implementing a substitution cipher decryption routine in assembly, while accessing global variables declared in a C source file.

(a) Setting the Context

A **substitution cipher** is a method of encryption in which each character in the plaintext is replaced by a corresponding character defined by a fixed mapping. Given the mapping (**substitution array**) and the encrypted message (**ciphertext**), the original plaintext can be recovered by **reversing the substitution**.

(b) Problem Statement

You are given a C file containing two global variables:

1. **substitution** — A substitution array of size 26, where the element at index i represents the character that replaces the i^{th} letter of the alphabet during encryption.
For example, if `substitution[0] = 'q'`, it means the letter 'a' was encrypted as 'q'.
2. **ciphertext** — A null-terminated string representing the encrypted message.

Your task is to write a RISC-V assembly file that decrypts the ciphertext by reversing the substitution mapping and stores the recovered plaintext.

(c) Requirements

1. Your .s file must reference the global variables *substitution* and *ciphertext* as declared in the provided .c file.
2. For each character in the ciphertext, find its corresponding plaintext character by inverting the substitution array.
3. The decryption logic must be implemented **entirely in RISC-V assembly**.
4. You may assume all alphabetic characters in the ciphertext are lowercase.

Template codes

Listing 2: The .c code structure

```
#include<stdio.h>

extern int decrypt(...); // forward declaration with arguments
extern char ans[]; // forward declaration

char* cipher_text = "kocvuzwyluiwslylkmzlzvnclwylfbzlbok";
char substitution[] = {
    'w', 't', 'z', 'f', 'v',
    'i', 's', 'h', 'l', 'g',
    'a', 'y', 'e', 'r', 'b',
    'c', 'p', 'u', 'k', 'm',
    'o', 'd', 'x', 'n', 'j',
    'q'
};

int main(){
    // call the decryption function from assembly
    printf("recovered_plaintext:_%s\n", ans);
}
```

Listing 3: The .asm code structure

```
.section .data
    .global ans # need to declare as a global variable
    ans: .space 40

.section .text
.global decrypt # need to declare as a global variable

decrypt:
    # perform decryption over the string cipher_text
    ret
```

Compilation and Execution

To compile your assembly program with the provided linker script and run it using the Spike simulator in debug mode, use the following commands:

```
# Compile the program
$ riscv64-unknown-elf-gcc <your_asm_code>.s <your_c_code>.c
```

```
# Execute using Spike in debug mode
$ spike -d pk program.out
```

Submissions

- Submit your assembly (.s) and C code (.c) files
- A screenshot showing the printed deciphered string

Problem 3: Process context switching and timer interrupts (50 Points)

Objective

To implement a multi-process environment in Machine-mode (M-mode) using timer interrupts for context switching. Students will develop code in C and assembly to context switch by saving and restoring process states.

Kernel Components to Implement

The implementation is divided into several key assembly and C components that together form a basic multitasking kernel.

1. `_start` (Assembly Entry Point)

This is the first code executed. Your tasks here are:

- Initialize the `sp` (Stack Pointer) to the `_machine_stack`.
- Call `setup_processes` to initialize the process table.
- Use the return values to set the first process's `mepc` and `sp`.
- Configure `mtvec`, `mie`, `mstatus` and `mtimecmp` (0x100 initially).

2. Timer Mechanism: `mtime` and `mtimecmp`

The RISC-V timer interrupt is triggered when the value of the `mtime` register (which increments at a fixed frequency) becomes greater than or equal to the value in the `mtimecmp` register.

- **increment_timer:** Inside the trap handler, you must update the timer to trigger the next interrupt. For this lab, increment the current value of `mtimecmp` by **0x100** in the `increment_timer` function.
- **Useful Note:** The `mtimecmp` and `mtime` are memory mapped registers and are mapped to addresses 0x2004000 and 0x200bff8.

3. `mttrap_handler` (Assembly)

This is the core of your context switcher. When a timer interrupt occurs:

- **Save Context:** Save all registers (`x1`, `x3-x31`) onto the *current process's* stack.
- **Switch:** Update `mtimecmp` via `increment_timer`. Then, call `switch_processes` to get the next process's state.
- **Restore Context:** Load the registers from the *new process's* stack, set `mepc`, and execute `mret`.

4. setup_processes and switch_processes (C)

These functions manage the logic of the process table:

- **setup_processes**: Assign PIDs, partition the 4KB user stack space into 1KB chunks for each process, and set initial Program Counters.
- **switch_processes**: Implement Round-Robin scheduling (1->2->3->4->1) by modifying the process table and returning the next process's data.

User Processes (ucode)

To verify your kernel, you will run the following user-mode codes concurrently:

1. **Register Integrity Checker (ucode_checker)**: This code fills registers s1-s11, t0-t6, and a0-a7 with the value 11. It continuously loops to check if these values have changed. A change indicates a failure in your context-saving logic. Do not take advantage the hardcoded 11 value to not implement proper register saving.
2. **Recursive Fibonacci (ucode_fib)**: This code calls a recursive C function. This stress-tests your register and **ra (return address) persistence**.
3. **Global Counter (ucode_counter)**: A simple process that increments a global variable.

Compilation and Execution

Listing 4: Compilation Command

```
$ riscv64-unknown-elf-gcc -mmodel=medany -nostartfiles \  
-T link.ld lab7_q3.s helper.c -o a.out  
  
# Execute using Spike in debug mode  
$ spike -d a.out
```

Useful Commands

Listing 5: Compilation Command

```
(spike) until pc 0 <instruction_address>  
(spike) mtime  
0x0000000000000012c  
(spike) mtimecmp 0  
0x00000000000000100  
# Spike help:- Stop when virtual memory <addr> in [core] (physical address <addr> if  
omitted) becomes <val>  
# Use this with the mtimecmp address (0x2004000)  
(spike) until mem <core> <addr> <val>
```

Submission Requirements

- Completed lab7_q3.s file.
- Completed helper.c with the setup and switching logic.
- Screenshots:

- Run spike until the first interrupt is raised to the handler (until the first time execution reaches `mtrap_handler`) and show the contents of `mtime` and `mtimecmp` at this point.
- Run spike until `mtimecmp` reaches `0xd00` using the previously mentioned command. If correctly done, this should stop in the `increment_timer` function. Dump the value of `count_var`, `flag_var` and `fib_array` at this point.
- Run `until mem 0 0x2004000 X` for $X = 0x200, 0x300, 0x400, 0x500$. Dump the contents of `current_process` for each.
- Run `until mem 0 0x2004000 0x300`. Dump the contents of `process_table[0]`. This can be done by finding the address of the array and dumping 4 consecutive '64 bit' integers which represent the values.

Submission Note

Ensure the submission items are properly organized. Place all submittables for question 1 under a folder named Q1, submittables for question 2 under Q2 and submittables for question 3 under Q3.